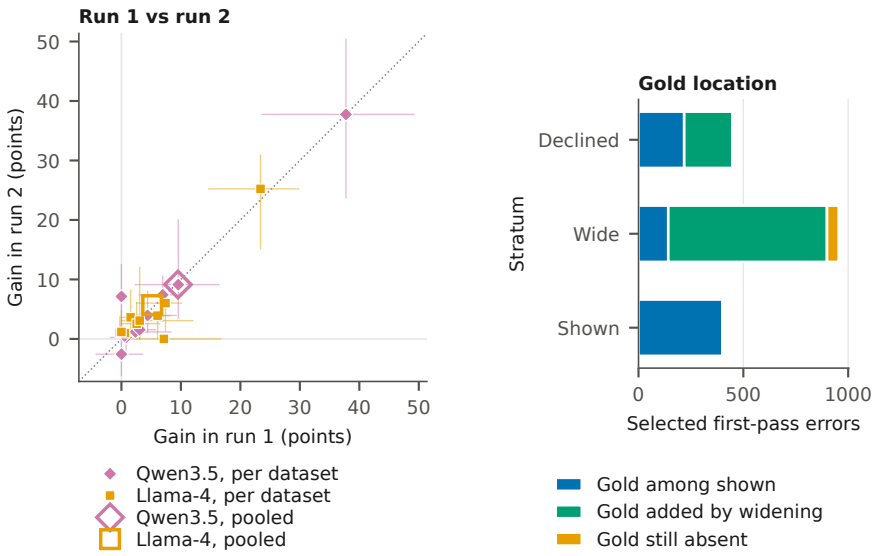


Figure 10: Gain per dataset in two runs of Llama-4 and Qwen3.5 under the margin rule, with pooled points (a), and the selected first-pass errors by gold location on the Margin-Reflect second pass (b).

Minus	Change [95% CI]	Holm p
No reflection	$+5.41$ [$+2.98$, $+8.75$]	0.0025
Random selection	$+3.13$ [$+1.69$, $+5.16$]	0.0025
Reflect on all	-0.15 [-0.25, -0.07]	0.0025
Declined only	$+2.95$ [$+1.50$, $+4.96$]	0.0025
Full candidate list	$+2.68$ [$+0.94$, $+4.97$]	0.0025

Table 23: Holm-adjusted comparisons: Margin-Reflect, on its own second pass, minus each of five selection rules, pooled over field-model pairs on the core test set, with 95% intervals and Holm-adjusted p. Each rule that reflects uses Margin-Reflect's second pass, with random selection at Margin-Reflect's reflection rate.

P The Margin-Reflect Configuration

Margin-Reflect shares Map-Reflect's first pass and widening rule but differs in both selection and reflection. It selects every field whose score margin is at most a threshold t_m fitted per model on development data (Table 9), and it reflects with a neutral prompt that adds evidence and a pairwise old-versus-new check, capping each description at a fixed length (prompts in Appendix Q). Table 22 places it beside the selection rules of Table 11, which separates its two differences from Map-Reflect. At the frozen values, its pooled development accuracy is 93.9%, against 91.2% for no reflection. Figure 10 charts its run-to-run agreement and the gold location of its selected errors, and Figures 7 and 8 chart further diagnostics of both configurations. Figure 8a and b both use the Margin-Reflect second pass: they show how its development accuracy changes as t_m varies, and its test change when its thresholds are refit on a smaller sample of the development data.

Holm-Adjusted Comparisons. We test the contrasts of Margin-Reflect against five selection rules on its own second pass as one family with a Holm adjustment, and each of the five contrasts is distinguishable from zero, with every adjusted p at the resolution floor of the bootstrap (Table 23). Margin-Reflect is more accurate than four of the rules and slightly less accurate than reflection on all fields. The rule *declined only* widens and reflects on the declined fields alone, and the *full candidate list* shows the first pass every field of the target type; its contrast covers 12,612 field-model pairs, since NL2GQL has no run on the full candidate list.

Gold Location and Run-to-Run Agreement. In this configuration, too, most observed repairs occur where widening adds a gold target, and Figure 10b counts its selected first-pass errors by where their gold target lies. Two runs of Llama-4 and Qwen3.5 under the margin rule, with thresholds fitted on unrounded margins, give pooled gains over the first pass of $+7.34$ and $+7.41$ points (Figure 10a).

Q Prompts

All but one of the prompts below are shown on one worked example: the source field `monitor_Alert.severity` and the four fields of an incident type, the same case as Figure 1. The

first-pass prompt lists the four fields; the reflection prompts use the top three of its ranking, and the verifier and check prompts its leading two, the first answer and the proposal. The exception is one Map-Reflect verifier on a MIMIC-OMOP field, shown to illustrate long field descriptions. The Self-Refine refinement prompt shows a placeholder where the model's feedback goes. The system and user messages are passed to each model's chat template. Map-Reflect uses the reflection with symmetric evidence and the either-order verifier below; the Margin-Reflect configuration (Appendix P) uses the neutral reflection with evidence and the pairwise check. The remaining prompts are the other second-pass reflection strategies we compare.

First pass: ranking

```
[system]
You are an expert in API schema mapping.

[user]
You are given a MONITOR property and a list
  of candidate ITSM properties.
Rank the candidates by how well they
  semantically match the MONITOR property.

MONITOR property: monitor_Alert.severity
Description: How serious the alert is.

Candidates:
1. id: itsm_Incident.criticalityLevel
   text: itsm_Incident.criticalityLevel --
     Business criticality of the affected
     service.
   distance: 0.5000
2. id: itsm_Incident.priority
   text: itsm_Incident.priority -- Priority
     assigned to the incident, from P1 to
     P4.
   distance: 0.5000
3. id: itsm_Incident.status
   text: itsm_Incident.status -- Lifecycle
     state of the incident.
   distance: 0.5000
4. id: itsm_Incident.openedAt
   text: itsm_Incident.openedAt -- When the
     incident was opened.
   distance: 0.5000

Task: Produce a JSON array of objects,
  ordered from best match to worst, where
  each object has:
- "id": candidate id (string, exactly as
  shown above)
- "score": a float between 0.0 and 1.0
  (higher is better match)
- "reason": one concise sentence explaining
  why you assigned that score

Return only the JSON array and nothing
  else.
```

Map-Reflect: reflection with symmetric evidence

```
[system]
You are an expert schema mapper reviewing
  your own work.

[user]
You previously ranked candidate itsm
  properties for a monitor property.

monitor property: monitor_Alert.severity
   entity: Alert (in monitor)
   meaning: How serious the alert is.
   data type: String
   neighbouring fields: message, raisedAt

Candidates:
1. id: itsm_Incident.criticalityLevel
   type: Int
   text: itsm_Incident.criticalityLevel --
     Business criticality of the affected
     service.
2. id: itsm_Incident.priority
   type: String
   text: itsm_Incident.priority -- Priority
     assigned to the incident, from P1 to
     P4.
3. id: itsm_Incident.status
   type: String
   text: itsm_Incident.status -- Lifecycle
     state of the incident.

Your previous ranking (best first):
1. itsm_Incident.criticalityLevel - Both
  describe how serious the issue is.
2. itsm_Incident.priority - Priority is set
  from the severity of the alert.
3. itsm_Incident.status - Status is the
  lifecycle state, not seriousness.

The rule of this task: map the source field
  to the target field that holds the same
  data about the same entity. Several
  source fields may map to the same target
  field, and more than one target field can
  be acceptable.

Evidence for the leading candidates (the
  same facts for each, listed
  alphabetically):
- itsm_Incident.criticalityLevel
   entity: Incident (in itsm)
   meaning: Business criticality of the
     affected service.
   data type: Int
   neighbouring fields: priority, status,
     openedAt
- itsm_Incident.priority
   entity: Incident (in itsm)
   meaning: Priority assigned to the
     incident, from P1 to P4.
   data type: String
   neighbouring fields: criticalityLevel,
     status, openedAt
- itsm_Incident.status
   entity: Incident (in itsm)
   meaning: Lifecycle state of the
     incident.
   data type: String
```

```
   neighbouring fields: criticalityLevel,
     priority, openedAt

Review your previous ranking against the
  descriptions and the evidence. Compare
  the leading candidates on the same
terms: the entity each belongs to, its
  meaning, its data type (with units and
  allowed values where the description
gives them) and its neighbouring fields.
  Keep your previous top answer unless you
  find a concrete reason it is wrong,
such as a different meaning, data type or
  entity than the source property. Do not
  change it only because another
candidate has a similar name.

Return a JSON object: {"analysis": "one or
  two sentences", "revised_ranking":
  [{"id": "...", "score": 0.0, "reason":
  "..."}, ...]}
with ids exactly as shown, best first.
```

Map-Reflect: either-order verifier, run with both option orders

```
[system]
You are an expert schema mapper.

[user]
Which target property is the correct
  mapping for this monitor property?

The rule of this task: map the source field
  to the target field that holds the same
  data about the same entity. Several
  source fields may map to the same target
  field, and more than one target field can
  be acceptable.

monitor property: monitor_Alert.severity
   entity: Alert (in monitor)
   meaning: How serious the alert is.
   data type: String
   neighbouring fields: message, raisedAt

Option 1: itsm_Incident.criticalityLevel
   entity: Incident (in itsm)
   meaning: Business criticality of the
     affected service.
   data type: Int
   neighbouring fields: priority, status,
     openedAt

Option 2: itsm_Incident.priority
   entity: Incident (in itsm)
   meaning: Priority assigned to the
     incident, from P1 to P4.
   data type: String
   neighbouring fields: criticalityLevel,
     status, openedAt

Judge both options on the same terms: the
  entity each belongs to, its meaning, its
  data type (with units and allowed
values where the description gives them)
  and its neighbouring fields, not name
  similarity. Return a JSON object:
{"choice": 1 or 2, "reason": "one
  sentence"}
```

The verifier is called twice, once as shown, with the first answer as option 1, and once with options 1 and 2 swapped, so that the proposal is option 1; the two calls are otherwise identical.

Map-Reflect's reflection and verifier prompts show each field's own definition and ETL notes first, then each table's shared description once, in full. In the worked example each description is one short sentence with no table text. On MIMIC-OMOP, where descriptions are long, the verifier for `CHARTEVENTS.CHARTTIME`, with the first answer `omop_MEASUREMENT__measurement_time` as option 1, reads:

Map-Reflect with full field descriptions: MIMIC-OMOP verifier

```
[system]
You are an expert schema mapper.

[user]
Which target property is the correct
  mapping for this mimic_CHARTEVENTS
  property?

The rule of this task: map the MIMIC column
  to the OMOP CDM column that receives its
  value in the ETL (the value may be
  reformatted or recoded). Several source
  columns may feed the same OMOP column.

mimic_CHARTEVENTS property:
  mimic_CHARTEVENTS__CHARTTIME
   entity: CHARTEVENTS (in mimic)
   meaning: records the time at which an
     observation was made
   data type: DATE
   neighbouring fields: SUBJECT_ID,
     HADM_ID, ICUSTAY_ID, ITEMID,
     STORETIME, CGID

Table context (shared by every property of
  a table, given once per table):
- CHARTEVENTS (mimic): Contains all charted
  data for all patients.
- MEASUREMENT (omop): The MEASUREMENT table
  contains records of Measurements, i.e.
  structured values (numerical or
  categorical) obtained through systematic
  and standardized examination or testing
  of a Person or Person's sample. The
  MEASUREMENT table contains both orders
  and results of such Measurements as
  laboratory tests, vital signs,
  quantitative findings from pathology
  reports, etc. Measurements are stored as
  attribute value pairs, with the attribute
  as the Measurement Concept and the value
  representing the result. The value can be
  a Concept (stored in VALUE_AS_CONCEPT),
  or a numerical value (VALUE_AS_NUMBER)
  with a Unit (UNIT_CONCEPT_ID). The
  Procedure for obtaining the sample is
  housed in the PROCEDURE_OCCURRENCE table,
  though it is unnecessary to create a
  PROCEDURE_OCCURRENCE record for each
```

```
  measurement if one does not exist in the
  source data. Measurements differ from
  Observations in that they require a
  standardized test or some other activity
  to generate a quantitative or qualitative
  result. If there is no result, it is
  assumed that the lab test was conducted
  but the result was not captured.

Option 1:
  omop_MEASUREMENT__measurement_time
   entity: MEASUREMENT (in omop)
   meaning: ETL: This is present for
     backwards compatibility and will be
     deprecated in an upcoming version.
   data type: varchar(10)
   neighbouring fields: measurement_id,
     person_id, measurement_concept_id,
     measurement_date,
     measurement_datetime,
     measurement_type_concept_id

Option 2:
  omop_MEASUREMENT__measurement_datetime
   entity: MEASUREMENT (in omop)
   meaning: ETL: This is not required,
     though it is in v6. If a source does
     not specify datetime the convention is
     to set the time to midnight
     (00:00:0000)
   data type: datetime
   neighbouring fields: measurement_id,
     person_id, measurement_concept_id,
     measurement_date, measurement_time,
     measurement_type_concept_id

Judge both options on the same terms: the
  entity each belongs to, its meaning, its
  data type (with units and allowed
values where the description gives them)
  and its neighbouring fields, not name
  similarity. Return a JSON object:
{"choice": 1 or 2, "reason": "one
  sentence"}
```

Margin-Reflect configuration: reflection, neutral + evidence

```
[system]
You are an expert schema mapper reviewing
  your own work.

[user]
You previously ranked candidate itsm
  properties for a monitor property.

monitor property: monitor_Alert.severity
Description: How serious the alert is.

Candidates:
1. id: itsm_Incident.criticalityLevel
   type: Int
   text: itsm_Incident.criticalityLevel --
     Business criticality of the affected
     service.
2. id: itsm_Incident.priority
   type: String
   text: itsm_Incident.priority -- Priority
     assigned to the incident, from P1 to
     P4.
3. id: itsm_Incident.status
   type: String
   text: itsm_Incident.status -- Lifecycle
     state of the incident.

Your previous ranking (best first):
1. itsm_Incident.criticalityLevel
  (score=0.62) - Both describe how serious
  the issue is.
2. itsm_Incident.priority (score=0.58) -
  Priority is set from the severity of the
  alert.
3. itsm_Incident.status (score=0.2) -
  Status is the lifecycle state, not
  seriousness.

Evidence you did not have in your first
  ranking:
Source monitor_Alert.severity: type String;
  belongs to Alert with fields message,
  raisedAt
Candidate itsm_Incident.criticalityLevel:
  type Int; belongs to Incident with fields
  priority, status, openedAt
Candidate itsm_Incident.priority: type
  String; belongs to Incident with fields
  criticalityLevel, status, openedAt
Candidate itsm_Incident.status: type
  String; belongs to Incident with fields
  criticalityLevel, priority, openedAt
One-to-one conflicts: none

Review your previous ranking against the
  descriptions and the evidence. Keep your
  previous top answer unless you find a
concrete reason it is wrong, such as a
  different meaning, data type or entity
  than the source property.
Do not change it only because another
  candidate has a similar name.

Return a JSON object: {"analysis": "one or
  two sentences", "revised_ranking":
  [{"id": "...", "score": 0.0, "reason":
  "..."}, ...]}
with ids exactly as shown, best first.
```

Margin-Reflect configuration: pairwise check

```
[system]
You are an expert schema mapper.

[user]
Which target property is the correct
  mapping for this monitor property?

monitor property: monitor_Alert.severity
Description: How serious the alert is.
  [type String]

Option 1: itsm_Incident.criticalityLevel --
  Business criticality of the affected
  service. [type Int; in Incident with
  priority, status, openedAt]
Option 2: itsm_Incident.priority --
  Priority assigned to the incident, from
  P1 to P4. [type String; in Incident with
  criticalityLevel, status, openedAt]

Compare meaning, data type and entity, not
```

```
  name similarity. Return a JSON object:
{"choice": 1 or 2, "reason": "one
  sentence"}
```

Strategy: doubt, a deliberately change-seeking control

```
[system]
You are an expert schema mapper reviewing
  your own work.

[user]
You previously ranked candidate itsm
  properties for a monitor property.

monitor property: monitor_Alert.severity
Description: How serious the alert is.

Candidates:
1. id: itsm_Incident.criticalityLevel
   text: itsm_Incident.criticalityLevel --
     Business criticality of the affected
     service.
2. id: itsm_Incident.priority
   text: itsm_Incident.priority -- Priority
     assigned to the incident, from P1 to
     P4.
3. id: itsm_Incident.status
   text: itsm_Incident.status -- Lifecycle
     state of the incident.

Your previous ranking (best first):
1. itsm_Incident.criticalityLevel
  (score=0.62) - Both describe how serious
  the issue is.
2. itsm_Incident.priority (score=0.58) -
  Priority is set from the severity of the
  alert.
3. itsm_Incident.status (score=0.2) -
  Status is the lifecycle state, not
  seriousness.

Your previous top answer is likely wrong:
  it probably matches by name similarity
  instead of meaning. Assume it
is incorrect, look for the candidate that
  represents the same data, and change your
  top answer. Be bold.

Return a JSON object: {"analysis": "one or
  two sentences", "revised_ranking":
  [{"id": "...", "score": 0.0, "reason":
  "..."}, ...]}
with ids exactly as shown, best first.
```

Strategy: Self-Refine (feedback)

```
[system]
You are an expert schema mapper.

[user]
You previously ranked candidate itsm
  properties for a monitor property.

monitor property: monitor_Alert.severity
Description: How serious the alert is.

Candidates:
1. id: itsm_Incident.criticalityLevel
   text: itsm_Incident.criticalityLevel --
     Business criticality of the affected
     service.
2. id: itsm_Incident.priority
   text: itsm_Incident.priority -- Priority
     assigned to the incident, from P1 to
     P4.
3. id: itsm_Incident.status
   text: itsm_Incident.status -- Lifecycle
     state of the incident.

Your previous ranking (best first):
1. itsm_Incident.criticalityLevel
  (score=0.62) - Both describe how serious
  the issue is.
2. itsm_Incident.priority (score=0.58) -
  Priority is set from the severity of the
  alert.
3. itsm_Incident.status (score=0.2) -
  Status is the lifecycle state, not
  seriousness.

Review your previous answer and find
  problems with your answer. Answer in at
  most five sentences.
```

Strategy: Self-Refine (refine)

```
[system]
You are an expert schema mapper.

[user]
You previously ranked candidate itsm
  properties for a monitor property.

monitor property: monitor_Alert.severity
Description: How serious the alert is.

Candidates:
1. id: itsm_Incident.criticalityLevel
   text: itsm_Incident.criticalityLevel --
     Business criticality of the affected
     service.
2. id: itsm_Incident.priority
   text: itsm_Incident.priority -- Priority
     assigned to the incident, from P1 to
     P4.
3. id: itsm_Incident.status
   text: itsm_Incident.status -- Lifecycle
     state of the incident.

Your previous ranking (best first):
1. itsm_Incident.criticalityLevel
  (score=0.62) - Both describe how serious
  the issue is.
2. itsm_Incident.priority (score=0.58) -
  Priority is set from the severity of the
  alert.
3. itsm_Incident.status (score=0.2) -
  Status is the lifecycle state, not
  seriousness.

Review your previous answer and find
  problems with your answer.

Your feedback on your previous answer:
<the model's feedback from the previous
  call>

Based on the problems you found, improve
  your answer.
```

```
Return a JSON object: {"analysis": "one or
  two sentences", "revised_ranking":
  [{"id": "...", "score": 0.0, "reason":
  "..."}, ...]}
with ids exactly as shown, best first.
```

Strategy: neutral

```
[system]
You are an expert schema mapper reviewing
  your own work.

[user]
You previously ranked candidate itsm
  properties for a monitor property.

monitor property: monitor_Alert.severity
Description: How serious the alert is.

Candidates:
1. id: itsm_Incident.criticalityLevel
   text: itsm_Incident.criticalityLevel --
     Business criticality of the affected
     service.
2. id: itsm_Incident.priority
   text: itsm_Incident.priority -- Priority
     assigned to the incident, from P1 to
     P4.
3. id: itsm_Incident.status
   text: itsm_Incident.status -- Lifecycle
     state of the incident.

Your previous ranking (best first):
1. itsm_Incident.criticalityLevel
  (score=0.62) - Both describe how serious
  the issue is.
2. itsm_Incident.priority (score=0.58) -
  Priority is set from the severity of the
  alert.
3. itsm_Incident.status (score=0.2) -
  Status is the lifecycle state, not
  seriousness.

Review your previous ranking against the
  descriptions. Keep your previous top
  answer unless you find a
concrete reason it is wrong, such as a
  different meaning, data type or entity
  than the source property.
Do not change it only because another
  candidate has a similar name.

Return a JSON object: {"analysis": "one or
  two sentences", "revised_ranking":
  [{"id": "...", "score": 0.0, "reason":
  "..."}, ...]}
with ids exactly as shown, best first.
```

Strategy: check after neutral

```
[system]
You are an expert schema mapper.

[user]
Which target property is the correct
  mapping for this monitor property?

monitor property: monitor_Alert.severity
Description: How serious the alert is.

Option 1: itsm_Incident.criticalityLevel --
  Business criticality of the affected
  service.
Option 2: itsm_Incident.priority --
  Priority assigned to the incident, from
  P1 to P4.

Compare meaning, data type and entity, not
  name similarity. Return a JSON object:
{"choice": 1 or 2, "reason": "one
  sentence"}
```

Description Length. In Map-Reflect, widening reuses these prompts on the enlarged list, and the verifier runs in both orders. Map-Reflect's prompts show full descriptions and the correspondence rule, while the prompts of the Margin-Reflect configuration (Appendix P) add a line of one-to-one conflicts, the other source fields of the sample whose previous answer is the same target, as a hint that does not forbid shared targets. A fixed cap on each description's length applies to that configuration's reflection and check, while every reflection strategy of Table 5 shows full descriptions. The stated-confidence and P(True) values in our tables are computed on prompts with full descriptions, and the Prompt-Matcher baseline of Table 1 shows full descriptions. The cap shortens many descriptions on the three OMOP datasets, MIMIC-OMOP and SchemaNet, where a column's description begins with its table's text, and a few on ITOpsGraph, Bio-ML, SemTab and OAEI Conference. We do not reproduce the other prompts, such as those of the paid confidence estimates, here.